

Running LLMs on an 8GB Jetson

The 25 most important findings and settings — distilled from a year of edge-AI projects

1. Turn the GUI off before running any model.

The desktop eats about 1.5GB of the 8GB. Run **sudo systemctl stop gdm.service** first; this single step decides whether a 4B model fits.

2. Always launch with unified memory enabled.

Set **GGML_CUDA_ENABLE_UNIFIED_MEMORY=1** on every launch. Without it the GPU cannot use system RAM and anything over ~2GB fails to load.

3. The no-pinned-memory trick frees half a gigabyte.

Adding **GGML_CUDA_NO_PINNED=1** saves ~500MB with zero speed penalty. It is the single fix that makes 32K context possible on 8GB.

4. Budget about 4.5GB for the model, not 5.2GB.

The real pool is 7.4GB total minus the gateway (~1.5GB, it grows), system overhead, and CUDA buffers. Plan weights + KV + 400MB CUDA inside ~4.5-4.7GB.

5. Put ALL layers on the GPU.

Full offload with **-ngl 99** is about 3x faster than the old partial-offload advice. This holds even for hybrid Mamba models like Granite 4.0 on recent llama.cpp builds.

6. The KV cache has exact math: know it before you launch.

KV bytes per token = 2 x layers x kv_heads x head_dim. Multiply by context and by bytes per element (f16=2.0, q8_0=1.0, q4_0=0.5). A 3B model can use anywhere from 18KB to 197KB per token — a 10x swing.

7. Architecture matters more than parameter count.

Gemma-4-E2B (1 KV head, head_dim 512) uses 36KB/token. Phi-3-3.8B (32 KV heads, no GQA) uses 197KB/token. Same size class, wildly different context limits.

8. q8_0 KV cache is the right default.

Near-lossless quality, halves the cache cost. Community testing agrees: q8_0 for agent workloads; shrink the context window before going to q4_0 KV.

9. Surprisingly, q8_0 KV can beat q4_0 at 32K.

For Spark-X2.5-4B, q4_0 KV at 32K OOMs (fragmentation kills it mid-run) while the bigger q8_0 cache runs 12/12 tests at full speed. Bigger-but-predictable beats smaller-but-fragmented.

10. Fall back gracefully: context ladder 48K → 32K → 16K → 8K, KV ladder f16 → q8_0 → q4_0.

Try the best first; step down only on OOM. Most 2-3B models land at 32K q4_0 or q8_0.

11. Only two models do 48K at full f16 precision.

Llama-3.2-1B and Qwen3.5-2B (an SSM hybrid). Everything else needs KV quantization to go past 32K.

12. One model fits its full native 128K context: Granite 4.0-3B.

The hybrid Mamba2 design plus q4_0 KV squeezes 128K into 8GB — at 7.2 tok/s when the context is deep.
Best model for huge document ingestion.

13. Unsloth Dynamic (UD) quants fit where regular quants fail.

Gemma-4-E4B at Q4_K_XL-UD runs 32K f16; the regular QAT q4_0 of the same model cannot load at all.
Same size, smarter weight allocation.

14. If you still OOM at 32K, put the KV cache on the CPU.

Add **-nkvo -ub 256 -b 256**. It costs about 30% speed but rescues 2.5GB+ models at 32K. Note: Qwen3-4B (8 KV heads) fails even with this.

15. Use MTP speculative decoding — it doubles speed for free.

Multi-token prediction gave: Gemma-4-E4B 15.6 to 31.7 tok/s (+103%), E2B 23.7 to 39.1 (+65%),
Qwen3.5-4B 13.1 to 24.9 (+90%). Only Qwen3.5 and Gemma-4 models support it (check for nextn layers).

16. Best MTP settings: n-max 4, p-min 0.8.

On E4B, MTP n4 pmin0.8 hit 35 tok/s on code (2x baseline) with no quality regression. The conservative p-min prevents speculative misfires from slowing greedy decoding.

17. No MTP model? Use n-gram speculative decoding.

Works on ANY model, zero VRAM cost, reuses patterns the model already generated. Free speedup for repetitive outputs like HTML and JSON.

18. For tool calling, use greedy decoding.

temp 0.0, top_k 1, repeat_penalty 1.0 for BFCL tests and any agent tool use. Deterministic output maximizes function-call accuracy.

19. For creative work, relax the sampler.

temp ~0.6, top_k 20-40, top_p 0.9-0.95 won across a 27-model sweep. Per-category tuning beats one setting for everything (the routing-service approach).

20. Turn thinking mode OFF for models under 4B.

Small models loop endlessly in their own reasoning traces. /no_think or enable_thinking=false is mandatory; otherwise generations never stop.

21. The DRY sampler stops code repetition loops.

dry-multiplier 0.5, base 1.75, allowed-length 4 fixed endless-copy loops in game generation with Ornth.
Cheap insurance for any long generation.

22. Cap max_tokens below the context window.

The BFCL harness default (16384) overflows a 16K context and crashes the server. Keep prompt + max_tokens inside the window; ~12000 works at 16K.

23. Multi-turn tests reveal failures single-turn tests miss.

Models that pass a 12-prompt single-turn suite crash mid-BFCL (52 sequential multi-turn tests). Sequential long generations exhaust memory in ways one-shot tests never touch. Always run both.

24. Best models for agent tool-calling on 8GB (as of Oct 2026).

Granite 4.0-3B: BFCL 96.2%, 128K ctx — the champion. Gemma-4-E4B-UD: 92.3%, needs 32K q4_0 KV for full runs, 31.7 tok/s with MTP. Spark-X2.5-4B: 90.4%, the only 4B that survives a full 52-test BFCL at 32K q8_0.

25. Save results after every single test, and run one model at a time.

The 8GB environment OOMs unpredictably (gateway memory grows, kernel kills servers mid-run).

Incremental saves mean a crash costs one test, not the session. Two models at once is always an OOM.

Sources: granite4-jetson-hermes, 32k-context-llm-survey, e4b-mtp-benchmark, e4b-optimum-settings, jetson-model-zoo, llm-transmission-8gb-202609, ornith-q4km-retest, bfcl-jetson-test-v2. All numbers measured on Jetson Orin Nano 8GB, llama.cpp build 1051 (Oct 2026).